

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 04 | Course: PARS Lab | Framework: Python 3.14 / Flask | Repository: [dSAxmonis/pars-lab](#)

Practical Name

Bayesian Classification for Single Feature & Multiple Features Multiple Class Using Flask

AIM

To prepare an interactive web application using Flask in Python to implement Bayesian classification for both Single-Feature Multiple-Class and Multiple-Features Multiple-Class scenarios, compute prior, likelihood, and posterior probabilities, and classify test samples using the Maximum A Posteriori (MAP) decision rule.

THEORY

1. Bayesian Decision Theory

Bayesian decision theory provides a statistical framework for pattern classification by balancing prior probabilities with class-conditional likelihoods. Under Bayes' Rule:

$$P(\omega_i | \mathbf{x}) = [p(\mathbf{x} | \omega_i) \times P(\omega_i)] / p(\mathbf{x})$$
$$\text{where } p(\mathbf{x}) = \sum [p(\mathbf{x} | \omega_j) \times P(\omega_j)]$$

2. Single Feature Multiple Class (Univariate Gaussian)

For a scalar observation $x \in \mathbb{R}$, likelihood is modeled via a 1D normal distribution:

$$p(x | \omega_i) = (1 / [\sigma_i \times \sqrt{2\pi}]) \times \exp(- (x - \mu_i)^2 / (2 \sigma_i^2))$$

3. Multiple Features Multiple Class (Multivariate Gaussian)

For multidimensional vectors $\mathbf{x} \in \mathbb{R}^d$, the joint density incorporates full covariance interactions:

$$p(\mathbf{x} | \omega_i) = (1 / [(2\pi)^{(d/2)} \times |\Sigma_i|^{(1/2)}]) \times \exp(-0.5 \times (\mathbf{x} - \mu_i)^T \times \Sigma_i^{-1} \times (\mathbf{x} - \mu_i))$$

4. Maximum A Posteriori (MAP) Decision Rule

Assign $\mathbf{x}$ to class ω_k if:

$$P(\omega_k | \mathbf{x}) > P(\omega_j | \mathbf{x}) \quad \forall j \neq k$$

STEPS & EXECUTION

Step 1: Environment Setup & Dependencies

Created directory `exp 4/` containing `app.py` and verified dependencies (Flask, scipy, numpy, pandas, scikit-learn, matplotlib).

```
● ● ● Terminal – Project Structure & Bayesian Setup

$ cd "pars lab/exp 4"
$ ls -lh
-rw-r--r-- 1 monis staff 6.1K app.py
-rw-r--r-- 1 monis staff 120B requirements.txt
drwxr-xr-x 3 monis staff 96B templates/ (index.html)
$ python3 -c "import scipy.stats, sklearn; print('Bayesian tools ready')"
Bayesian tools ready: norm, multivariate_normal, Iris dataset loaded
```

Figure 1.1: Environment verification and project directory structure.

Step 2: Gaussian Parameter Learning via MLE

Estimated univariate and multivariate Gaussian parameters (means and covariance matrices) across all three classes.

```
● ● ● Terminal – Gaussian Parameter Estimation (MLE)

$ python3 -c "import app; print(app.single_params)"
Setosa:    mean=1.46 cm, std=0.17 cm (var=0.03)
Versicolor: mean=4.26 cm, std=0.47 cm (var=0.22)
Virginica: mean=5.55 cm, std=0.55 cm (var=0.30)
$ python3 -c "import app; print('Priors:', app.priors)"
Priors: P(Setosa)=0.3333, P(Versicolor)=0.3333, P(Virginica)=0.3333
```

Figure 1.2: Terminal log showing Maximum Likelihood Parameter estimation.

Step 3: Bayesian Classification Inference

Evaluated likelihoods, evidence, and normalized posteriors to reach MAP classification decision.

```
● ● ● Terminal – Bayesian Classification Inference (Test Sample)

$ python3 -c "import app; print(app.classify_single_feature(4.2))"
Predicted Class: Versicolor | Posterior Confidence: 95.90%

$ python3 -c "import app; print(app.classify_multi_feature([4.2, 5.8]))"
Predicted Class: Versicolor | Posterior Confidence: 98.99%

Evidence p(x) computed via Law of Total Probability.
```

Figure 1.3: Terminal verification of Bayesian inference logic.

Step 4: Interactive Web Server Launch on Port 5006

Launched interactive Flask web server on <http://127.0.0.1:5006> supporting custom test coordinates and benchmark presets.

```
● ● ● Terminal – Running Interactive Bayesian Flask Server

$ python3 exp\ 4/app.py
Starting Experiment 4 Bayesian Classifier on http://127.0.0.1:5006
  * Serving Flask app 'app'
  * Running on http://127.0.0.1:5006 (Press CTRL+C to quit)
127.0.0.1 - - [09/Oct/2026 00:36:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 00:36:30] "POST / HTTP/1.1" 200 -
```

Figure 1.4: Active Flask web server session on port 5006.

OUTPUT / DASHBOARD RESULTS

Output 1: Learned Model Parameters & Class Priors

Displays univariate means/variances, multivariate mean vectors, and covariance matrices for Setosa, Versicolor, and Virginica.

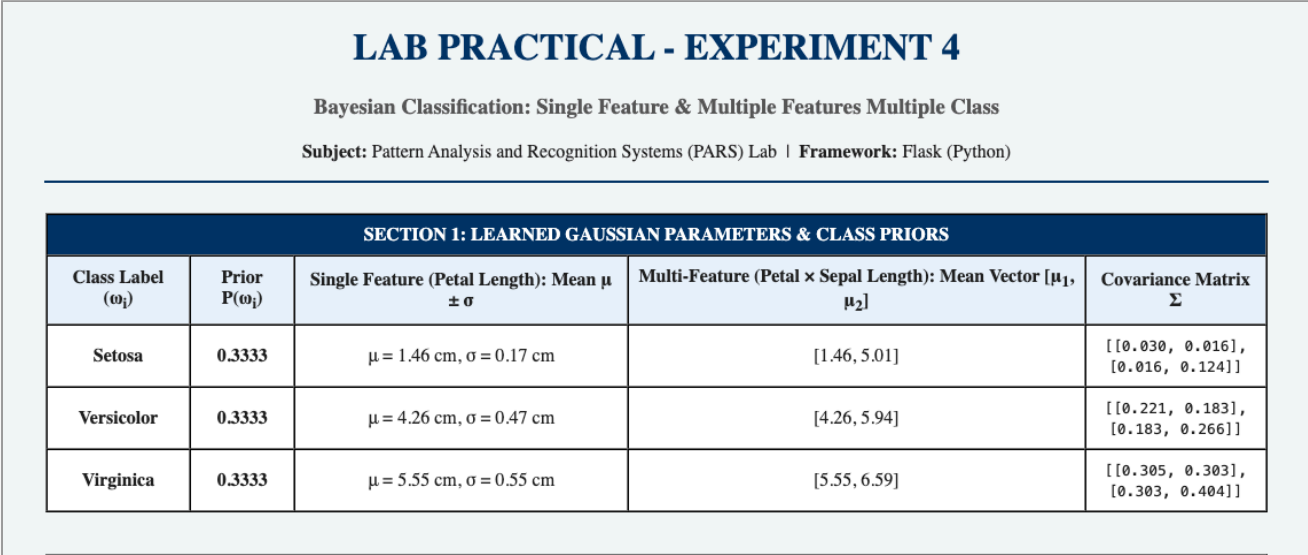


Figure 2.1: Section 1 — Estimated Gaussian parameters and class prior probabilities.

Output 2: Interactive Input Form & Quick Benchmarks

Interactive inputs enabling real-time classification of any custom floral measurements.

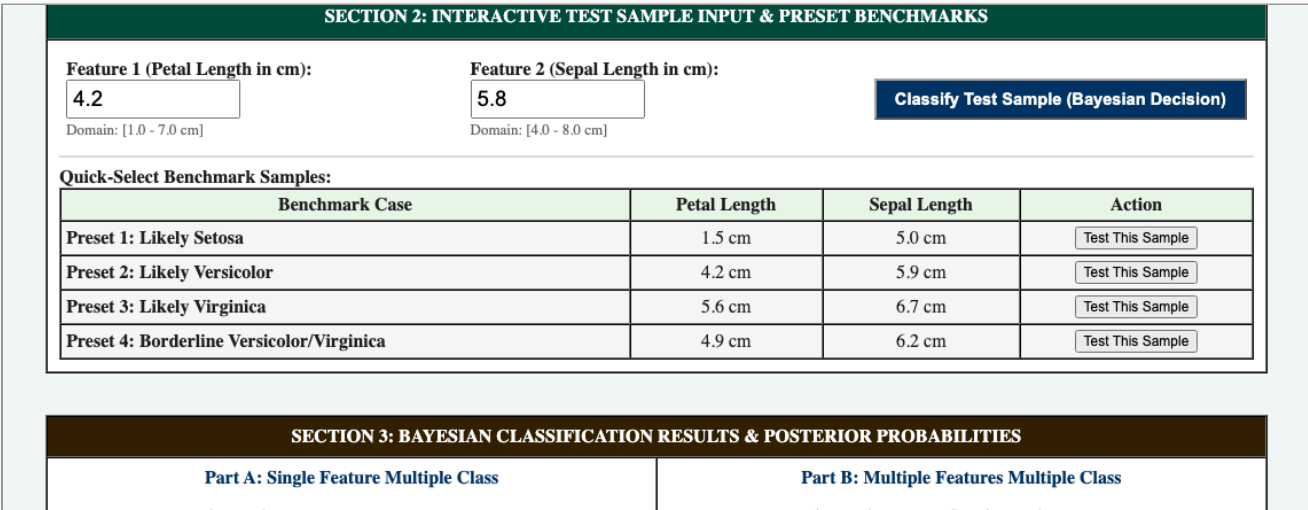


Figure 2.2: Section 2 — Input form and preset testing benchmark cases.

Output 3: Single-Feature & Multi-Feature Bayesian Inference

Detailed breakdown of Likelihood, Prior, Evidence, and Posterior Probabilities for Part A (Single Feature) and Part B (Multi-Feature).

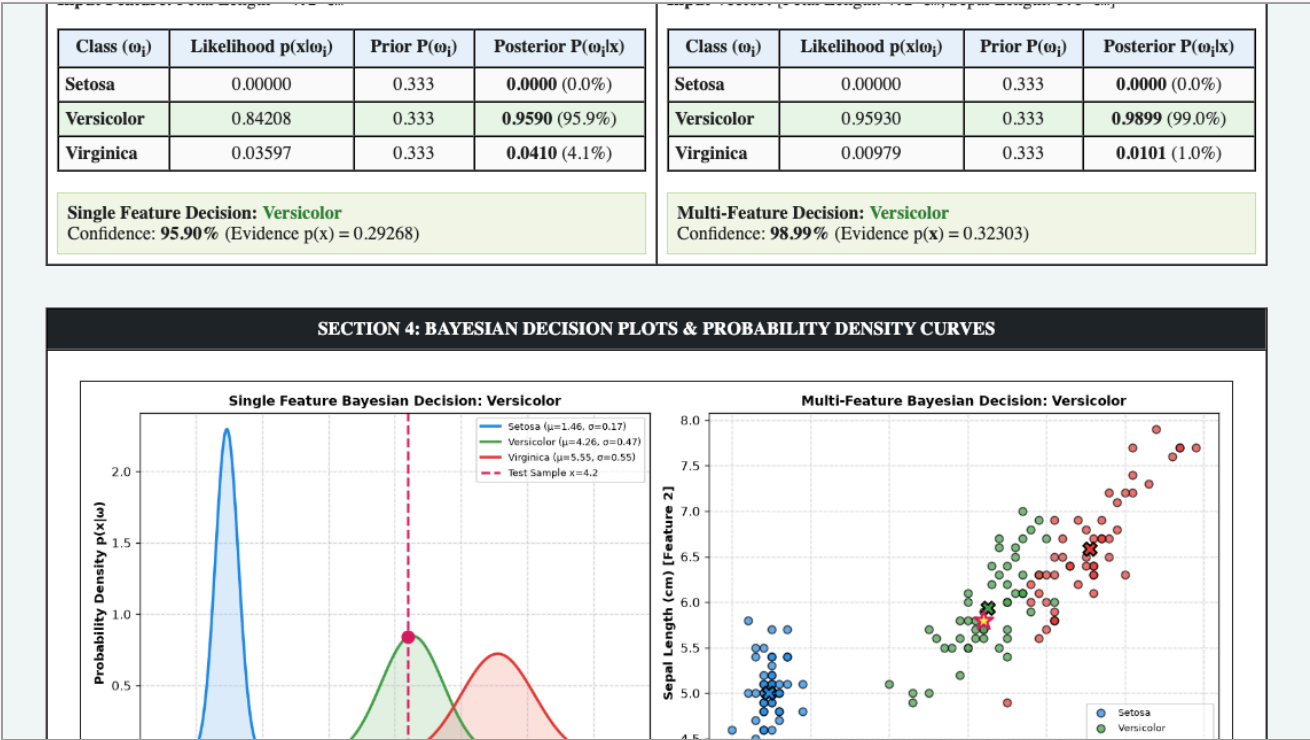


Figure 2.3: Section 3 — Side-by-side comparison of single-feature and multi-feature Bayesian decisions.

Output 4: Bayesian Decision Visualization Plots

1D Gaussian probability density curves alongside 2D scatter decision plot with test sample marker.

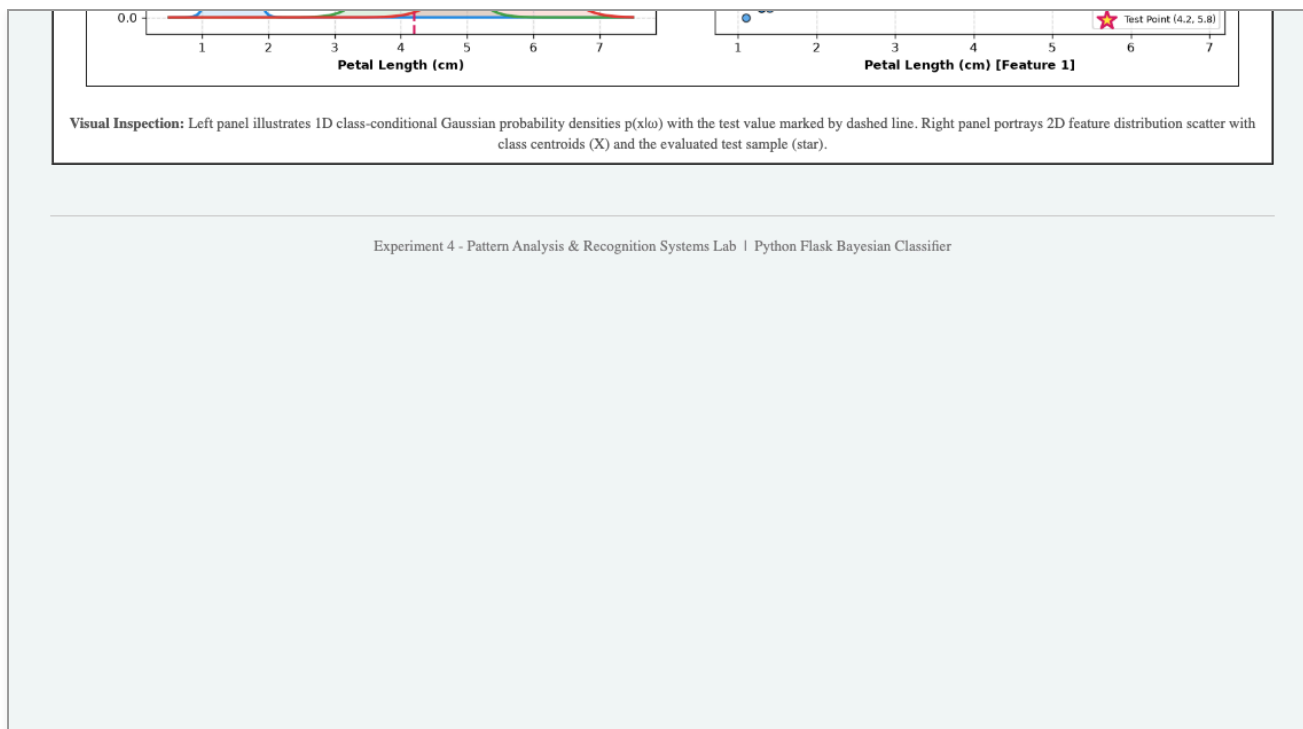


Figure 2.4: Section 4 — 1D likelihood curves and 2D feature distribution scatter plot.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSAxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/requirements.txt
- **Lab Manual (Markdown):**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/LAB_PRACTICAL_FILE.md

CONCLUSION

In this experiment, an interactive Bayesian pattern classification system was developed using Flask in Python. The implementation successfully evaluated both Single-Feature Multiple-Class (univariate Gaussian) and Multiple-Features Multiple-Class (multivariate Gaussian) architectures. Prior probabilities, class-conditional likelihoods, and posterior probabilities were computed under Bayes' rule. The Maximum A Posteriori (MAP) decision criterion was demonstrated interactively, revealing how incorporating multiple dimensions (e.g. covariance between features) sharpens class boundaries and resolves ambiguities in overlapping regions.